Programação de Sistemas Linux

Aula 09 — Processos e Daemons

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br

INSTITUTO FEDERAL
Santa Catarina
Câmpus Florianópolis

Notes

Abertura

Notes

Onde estamos

Aula 8 (feito)

- Dispositivos de caractere e bloco, `/dev` e udev
- `ioctl()`
- Estado do sistema via `/proc` e `/sys`
- **Avaliação 1**

Aula 9 (hoje)

- O que **é** um processo para o kernel
- `fork()`, `exec()` e `wait()`
- Zumbis e órfãos
- *Daemons*: o padrão *double fork*

Estrutura da aula

Hoje (2h): teoria e demonstrações. Próximo encontro (2h): **Laboratório 08**, no laboratório.



Notes

Objetivos da aula

Ao final desta aula, você será capaz de:

- ✓ Explicar o que o kernel guarda sobre cada processo e seus **estados**
- ✓ Usar `fork()` para criar um filho e `exec()` para trocar de programa
- ✓ Usar `wait()`/`waitpid()` e explicar processos **zumbis** e **órfãos**
- ✓ Explicar o que caracteriza um *daemon* e implementar o padrão *double fork* com `setsid()`, `umask()` e `chdir()`
- ✓ Usar **PID files** e `syslog()` para controlar e depurar um daemon

Regra de hoje

Todo comando que vocês já digitaram — `ls`, `gcc`, o próprio `bash` — nasceu de um `fork()` seguido de um `exec()`.



Notes

O que é um processo

Notes

Processo: mais que só “um programa rodando”

Definição

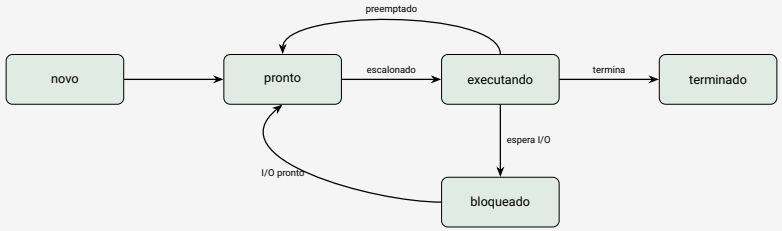
Um **processo** é uma instância de um programa em execução: seu código, seus dados, sua pilha, seu *heap* — e um conjunto de informações que o kernel mantém sobre ele, chamado **PCB** (*Process Control Block*): PID/PPID, estado, registradores, tabela de arquivos abertos, espaço de endereçamento e prioridade.

- O **mesmo** programa (`.c` compilado) pode virar **vários** processos independentes ao mesmo tempo — cada `./programa` é um novo processo
- O PCB é o que faz o kernel conseguir **pausar** um processo e retomar exatamente de onde parou



Notes

Estados de um processo



- **Pronto**: só falta a CPU ficar livre; **bloqueado**: esperando algo (disco, rede, teclado)
- Um processo **nunca** vai direto de bloqueado para executando — sempre passa por pronto de novo
- No `ps`, a coluna `STAT`: `R` (rodando/pronto), `S` (dormindo), `D` (I/O não-interruptível), `Z` (zumbi)



Notes

fork(): criando um processo filho

Notes

`fork()`: um vira dois

```c
pid_t pid = fork();

if (pid < 0) {
    perror("fork");          // fork falhou (raro: sem memoria/recursos)
} else if (pid == 0) {
    // este bloco roda so no FILHO
} else {
    // este bloco roda so no PAI; "pid" e o PID do filho
}

```

- `fork()` **duplica** o processo atual — depois dele, existem **dois** processos quase idênticos, ambos continuando do mesmo ponto do código
- A **única** diferença no código é o **valor de retorno**: `0` no filho, o PID do filho no pai



Notes

`fork()` em ação — e a pegadinha do buffer

```c
printf("Antes do fork: PID=%d\n", getpid());
fflush(stdout);              // sem isto, a linha acima sai DUAS vezes

pid_t pid = fork();
if (pid == 0) {
    printf("[FILHO]  PID=%d, PID do pai=%d\n", getpid(), getppid());
} else {
    printf("[PAI]    PID=%d, PID do filho=%d\n", getpid(), pid);
}

```

- O `printf` é **bufferizado**: se o texto ainda está no buffer da libc, o `fork()` **duplica o buffer junto** — e os dois processos imprimem a mesma linha
- **Depende de para onde a saída vai:** no terminal o buffer é **de linha** (o `\n` já descarrega, nada dobra); em arquivo ou *pipe* é **cheio** — e aí a linha sai **duas vezes**
- Mesmo mecanismo de *buffering* da Aula 5 (`printf` *vs.* `write` no `strace`), agora afetando **dois** processos



Notes

Copy-on-write: por que `fork()` é rápido

- `fork()` **parece** copiar toda a memória do processo — mas não copia, na prática
- As páginas de memória do pai e do filho ficam **compartilhadas**, marcadas como *somente leitura*
- Só quando **um dos dois escreve** numa página, o kernel finalmente faz a cópia **daquela página específica** — daí o nome *copy-on-write* (COW)

Por que isso importa

Um processo com 2 GB de memória pode dar `fork()` quase instantaneamente — a cópia de verdade só acontece sob demanda, página por página.



Notes

exec(): trocando de programa

Notes

`exec()`: mesma casca, programa novo

Definição

A família `exec()` **substitui** o código, dados e pilha do processo atual pelos de **outro** programa. O PID **não muda** — é o mesmo processo, mas agora rodando código diferente.

Variante	Diferença
`execl(caminho, arg0, arg1, ..., NULL)`	argumentos em **lista**
`execv(caminho, argv[])`	argumentos em **vetor**
`execlp` / `execvp`	procura o programa no `$PATH`
`execve`	permite passar o `environment` explicitamente



Notes

`fork()` + `exec()`: como o shell roda comandos

```c
pid_t pid = fork();

if (pid == 0) {
    execlp("ls", "ls", "-l", NULL);
    perror("execlp");   // so chega aqui se o exec FALHAR
    _exit(1);
} else {
    waitpid(pid, NULL, 0);
    printf("[PAI] filho (que virou 'ls') terminou\n");
}

```

- Se `execlp` funcionar, o código **depois** dele **nunca roda** — o processo virou outro programa
- Entre o `fork()` e o `exec()`, o filho pode **preparar o ambiente**: redirecionar `stdin`/`stdout` (é assim que `comando > arquivo` funciona), mudar de diretório, fechar descritores
- É **exatamente assim** que `bash` executa cada comando que vocês digitam



Notes

wait, zumbis e órfãos

Notes

`wait()` / `waitpid()`

```c
int status;
pid_t terminado = waitpid(pid, &status, 0);

if (WIFEXITED(status)) {
    printf("Filho %d saiu com codigo %d\n",
           terminado, WEXITSTATUS(status));
}

```

- `wait()` espera **qualquer** filho terminar; `waitpid(pid, ...)` espera um filho **específico**
- O `status` não é só o código de saída — `WIFEXITED`/`WEXITSTATUS` e macros parecidas decodificam ele
- Sem chamar `wait`/`waitpid`, o pai **nunca** recolhe esse status...

Notes

Processo zumbi: o filho que ninguém “enterrou”

```
$ ./zumbi_demo &
$ ps -eo pid,ppid,stat,cmd | grep zumbi_demo
17815 17453 S    ./zumbi_demo
17817 17815 Z    [zumbi_demo] <defunct>

```

Definição

Um **zumbi** é um processo que já **terminou**, mas cujo pai ainda **não chamou** `wait()/waitpid()`. O kernel mantém o PCB dele (com o status de saída) até que o pai o colete.

- Um zumbi não usa CPU nem memória de verdade — mas ocupa uma **entrada na tabela de processos**; muitos zumbis acumulados são um sintoma de bug



Notes

Processo órfão: o filho cujo pai já morreu

```c
pid_t pid = fork();
if (pid == 0) {
    sleep(2);
    printf("PID do pai apos o pai morrer: %d\n", getppid());
} else {
    printf("Pai termina ja, sem esperar o filho\n");
}

```

```
Pai termina ja, sem esperar o filho
PID do pai apos o pai morrer: 1

```

- Quando o pai termina **antes** do filho, o filho vira **órfão** — o kernel o *reparenta* para o processo `init`/reaper (normalmente PID 1)
- Diferente do zumbi: o órfão continua **rodando normalmente** — só mudou quem é o pai



Notes

Daemons: processos sem terminal

Notes

Daemon: um processo sem dono à vista

Definição

Um **daemon** é um processo que roda em **segundo plano**, **desvinculado** de qualquer terminal, geralmente iniciado no boot ou sob demanda, prestando um serviço (rede, log, agendamento…) para o resto do sistema.

- O nome vem da mitologia grega (um "espírito auxiliar" invisível) — e da convenção Unix de nomear muitos deles terminando em **d** (`sshd`, `crond`, `systemd`)
- Em `ps -eo pid,ppid,tty,cmd`, os daemons aparecem com `TTY = ?` (nenhum terminal) e `PPID = 1` (filhos diretos do `init`/`systemd`)
- Diferente de um programa comum, **ninguém está olhando** a tela dele — se ele travar ou precisar de atenção, só logs e sinais avisam

Notes

Três conceitos que faltavam

Conceito	O que é
Grupo de processos (PGID)	Conjunto de processos que recebem sinais juntos (ex.: `Ctrl+C` mata todo o pipeline)
Sessão (SID)	Conjunto de grupos de processos associados a um mesmo terminal de controle
Terminal de controle	O terminal que pode mandar `SIGINT`/`SIGHUP` para a sessão inteira

Um daemon precisa **sair** da sessão do terminal que o iniciou — senão, fechar aquele terminal manda `SIGHUP` para ele.



Notes

`setsid()`: criando uma sessão nova

```c
if (setsid() < 0) {
    perror("setsid");
    exit(1);
}

```

- Cria uma **nova sessão**, com o processo chamador virando líder — **sem** terminal de controle associado
- **Só funciona** se o processo chamador **não** for já um líder de grupo de processos — por isso o *primeiro* fork() do padrão double fork existe: garante que o filho **não** é líder de grupo



Notes

O padrão double fork

Notes

Por que *dois* `fork()`, e não um só?

O problema

Depois do `setsid()`, o processo é líder de **sessão**. Um líder de sessão **pode** voltar a adquirir um terminal de controle (por exemplo, ao abrir `/dev/tty`) — o que **não** é o que queremos de um daemon.

A solução: o segundo `fork()`

- O **neto** (filho do líder de sessão) **não** é líder de sessão
- Ele nunca pode readquirir um terminal de controle — está **definitivamente** livre



Notes

O padrão *double fork*, passo a passo

1. `fork()` — o **pai original** termina; o **filho** continua (já não é líder de grupo)
2. O filho chama `setsid()` — vira líder de uma **nova sessão**, sem terminal
3. O filho ignora `SIGHUP` (opcional, mas comum) e faz o **segundo** `fork()`
4. Esse filho intermediário **também termina**; o **neto** continua — ele é o daemon de fato
5. O neto ajusta `umask()`, `chdir("/")`, fecha/redireciona `stdin/stdout/stderr`

Por que terminar os processos intermediários

Cada `exit()` devolve o controle ao shell mais rápido **e** garante que o processo final seja reparentado para o `init/systemd` — exatamente o mecanismo dos processos órfãos que acabamos de ver.



Notes

Passos 1–3 em código

```c
pid_t pid = fork();                 /* 1o fork */
if (pid < 0)  exit(1);
if (pid > 0)  exit(0);               /* pai original termina */

if (setsid() < 0) exit(1);           /* 2: nova sessao, sem terminal */
signal(SIGHUP, SIG_IGN);             /* 3a: ignora SIGHUP */

pid = fork();                        /* 3b: 2o fork */
if (pid < 0)  exit(1);
if (pid > 0)  exit(0);               /* filho intermediario termina */

/* a partir daqui, estamos no NETO -- o daemon de verdade */

```



Notes

Passos 4–5 em código

```
1  umask(0027);          /* mascara de permissao previsivel p/ arquivos
       criados */
2  chdir("/");           /* nao prende nenhum ponto de montagem ocupado */
3
4  close(STDIN_FILENO);
5  close(STDOUT_FILENO);
6  close(STDERR_FILENO);
7  open("/dev/null", O_RDONLY);   /* vira fd 0 */
8  open("/dev/null", O_WRONLY);   /* vira fd 1 */
9  open("/dev/null", O_WRONLY);   /* vira fd 2 */
10
```

- `chdir("/")` evita que o daemon **prenda** um diretório — se ele tivesse ficado em, digamos, `/mnt/usb`, aquele disco não poderia ser desmontado
- Fechar/redirecionar os três descritores evita que o daemon escreva sem querer num terminal, ou trave esperando entrada que nunca vai vir



Notes

Controle, logging e systemd

Notes

PID file: como controlar um processo sem terminal

O problema

Sem terminal, você não pode simplesmente apertar `Ctrl+C`. Como mandar o daemon parar, ou saber se ele já está rodando?

```
1 FILE *f = fopen("/var/run/meudaemon.pid", "w");
2 fprintf(f, "%d\n", getpid());
3 fclose(f);
4
```

- Um script de controle lê o PID do arquivo e manda `kill -TERM $(cat pidfile)`
- Ao iniciar, o daemon pode **checar** se o PID file já existe e se aquele PID ainda está vivo — evita rodar **duas instâncias** sem querer



Notes

Logging: `printf` não serve, `syslog()` sim

Lembra dos três `close()` de agora pouco?

O daemon **não tem** `stdout` de verdade — ele está redirecionado para `/dev/null`. Um `printf` de depuração **desaparece**, sem erro nenhum.

```
1 openlog("meudaemon", LOG_PID, LOG_DAEMON);
2 syslog(LOG_INFO, "meudaemon iniciado, PID=%d", getpid());
3 syslog(LOG_ERR, "falha ao abrir arquivo: %s", strerror(errno));
4 closelog();
5
```

- Níveis: `LOG_ERR` (impede continuar), `LOG_WARNING` (inesperado, recuperável), `LOG_INFO` (eventos normais), `LOG_DEBUG` (depuração)
- Onde ler: `journalctl -t meudaemon` ou `/var/log/syslog`



Notes

Do daemon “manual” ao serviço `systemd`

Abordagem	Quem faz o double fork	Unit file
Daemon clássico (hoje)	O próprio programa em C	`Type=forking`
Daemon moderno	`systemd` (o programa **não** se destaca sozinho)	`Type=simple`

- Com `systemd`, `Type=simple` é o **padrão recomendado hoje em dia** — o systemd já cuida de isolar sessão/terminal, então o programa **nem precisa** fazer double fork
- `Type=forking` existe para dar suporte a daemons **clássicos**, escritos como o desta aula — o systemd espera o processo pai terminar e descobre o daemon de verdade pelo `PIDFile=`



Notes

Encerramento

Notes

Síntese da aula

Conceito	Pergunta que responde
PCB / estados	O que o kernel sabe sobre cada processo, e em que fase ele está?
`fork()` + COW	Como eu crio um processo filho, de forma barata?
`exec()`	Como um processo vira outro programa completamente diferente?
`wait`/zumbi/órfão	O que acontece quando um filho termina (ou o pai morre)?
Sessão / `setsid()`	Como um processo se desvincula do terminal que o iniciou?
Double fork	Como garantir que o daemon nunca readquira um terminal?
PID file	Como controlar um processo que não tem terminal?
`syslog()`	Como um daemon registra o que está acontecendo?

Todo `bash`, todo servidor, todo daemon do sistema é construído sobre essas peças.



Notes

Atividade prática — próximo encontro

Laboratório 08 — Processos e Daemons (2h)

Parte A, presencial:

1. `fork()` e a pegadinha do buffer do `printf`
2. `exec()`: trocando o programa em execução
3. Zumbis e órfãos, observados ao vivo com `ps`
4. Um **daemon completo** em C: *double fork*, PID file, `syslog`

Parte B, extraclasse: tarefas em paralelo com fork+exec+wait, script de controle `start/stop/status`, e registro como serviço `systemd` (`Type=forking`).

Atenção

No exercício do daemon, “meu programa roda” não basta: a correção olha a saída de `ps` (colunas `TTY`, `PPID`, `SID`) para confirmar que ele realmente se desvinculou do terminal.



Notes

Bibliografia de apoio

- Manuais: `man 2 fork`, `man 2 execve`, `man 2 waitpid`, `man 2 setsid`, `man 3 syslog`, `man 7 daemon`
- The Linux Programming Interface (Michael Kerrisk) — capítulos 24–26 e 37
- STEVENS, W. R.; RAGO, S. A. *Advanced Programming in the UNIX Environment* — capítulo sobre daemons

Notes

That's all folks!

INSTITUTO FEDERAL
Santa Catarina
Câmpus Florianópolis

Notes